

UMass Marketplace: Data Models and Module Diagrams

Team 10

Manager: Vidhita Mittal

Members: Vidhi Dave, Ngan Nguyen,
Gayatri Marathe, Arkar Myint

Table of contents

1. Frameworks

3. Function signatures

2. End point Routes

4. Testing Functions

Frameworks

Frameworks

- Front end

- UI/Prototyping: **Figma** for designing screens + iterating on user flows
- **TS + JS** for client side logic and interactivity
- **HTML/CSS** for structural layout + UI styling

- Backend/API

- **Express.js** (routing + controllers for listings, search/filter, seller profiles, messaging)
- **TypeScript**

Frameworks

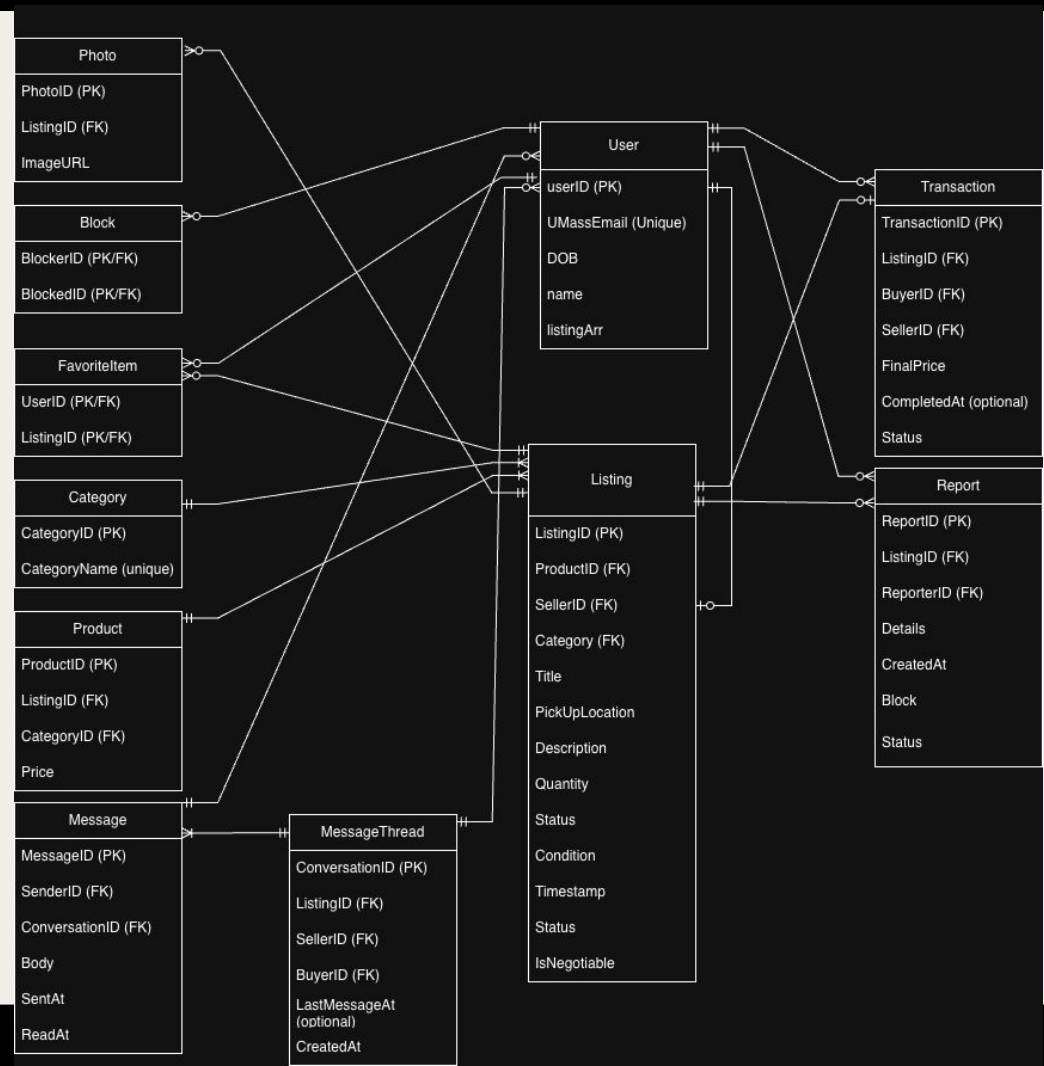
- ORM

- **TypeORM** (map database tables to code models)

- DBMS

- **MongoDB** to store marketplace data + relationships

Data Model Diagram



Endpoint Routes

API Routing

HTTP Method + URL Path = API Endpoint

e.g. GET `http://umassmarketplace:3000/api/listings/1025`

- URL (the address) and the HTTP Method (the action) work together to form a route.

HTTP Method + URL Path + Query Parameters = Filtered Request to an API Endpoint

e.g. GET `http://umassmarketplace:3000/api/listings?q=chair&maxPrice=100&status=active`

- Query parameters filter or modify the results
- Common for search or browse endpoints

Core Resource: /products

An example to show how a single resource (Products) handles the full life cycle of data (CRUD: Create, Read, Update, Delete)

Method	Endpoint	Description
POST	/products	Create a new product record
GET	/products	Retrieve all products
GET	/products/:id	Retrieve specific product details
PATCH	/products/:id	Update specific fields of a product
DELETE	/products/:id	Remove a product record

:id is a placeholder for real data (e.g. GET /products/1025)

Core Resource: /listings

Method	Endpoint	Description
POST	/listings	Create a new listing record
GET	/listings	Retrieve all listings
GET	/listings/:id	Retrieve specific listing details
GET	/listings/:id/?q=&category=&...	Retrieve specific fields of a listing record with query parameters
PATCH	/listings/:id	Update specific fields of a listing (title, price, status, etc.)
PATCH	/listings/:id/status	Update status of listings (sold/active/etc)
DELETE	/listings/:id	Remove a listing record

Function Signatures

What is a Function Signature?

A function signature includes:

- **Function name**
- **Input parameters**
- **Return type**

Example:

`createListing(data: ListingInput, userId: string): Promise<Listing>`

Layer	Example Signature
Controller	<code>createListing(req: Request, res: Response): Promise<void></code>
Service	<code>createListing(data: ListingInput, userId: string): Promise<Listing></code>
Model	<code>ListingModel.create(data): Promise<Listing></code>

Service layer returns data objects (e.g., Listing) while Controller layer transforms them into HTTP responses (e.g., JSON or status codes)

Input / Output Design

`getListingById(id: string): Promise<Listing | null>`

Input:

- id: string

Output:

- Listing if found
- null if not found

Request Flow

```
GET /listings/:id
↓
listingController.getListingById(req,res)
↓
listingService.getListingById(id)
↓
ListingModel.findById(id)
```

Main Marketplace Functions

Function Signature	Description
register(data: RegisterInput): Promise<AuthResponse>	Creates a new user account and returns authentication token
login(data: LoginInput): Promise<AuthResponse>	Authenticates a user and returns login token
createListing(data: ListingInput, userId: string): Promise<Listing>	Creates a new marketplace listing and returns the created listing so the client can immediately display it, confirm details, or update UI state without making another request.
getListings(query: ListingQuery): Promise<Listing[]>	Retrieves listings with optional filters
getListingById(id: string): Promise<Listing null>	Returns details of a specific listing
updateListing(id: string, data: ListingInput): Promise<Listing null>	Updates listing information
deleteListing(id: string): Promise<void>	Removes a listing from the marketplace

Supporting System Functions

Function Signature	Description
createProduct(data: ProductInput): Promise<Product>	Creates product record associated with listings
getProducts(): Promise<Product[]>	Retrieves all products
createCategory(data: CategoryInput): Promise<Category>	Creates a marketplace category
getCategories(): Promise<Category[]>	Returns all listing categories
createSeller(data: SellerInput): Promise<Seller>	Creates seller profile
getSellerById(id: string): Promise<Seller null>	Retrieves seller information

Testing Functions

Verify that core marketplace features work correctly before deployment.

We will test:

- Authentication
- Listings
- Search/filter
- Favorites
- Messaging
- Reports/Blocks

Testing approach:

- Unit test backend functions
- Test API endpoints with sample requests
- Check returned JSON / UI output
- Verify database changes in Postgres/pgAdmin after create, update, or delete actions

Component	Details
Feature Being Tested	Browse listings with search and category/price filters
Test Setup	Add sample listings to database: Mini Fridge (\$40, Appliances, ACTIVE), Desk Lamp (\$15, Dorm Essentials, ACTIVE), Refrigerator (\$120, Appliances, ACTIVE)
Action	GET /api/listings?q=fridge&category=Appliances&status=ACTIVE
Expected result	API returns status 200 OK and JSON: <pre>[{ "title": "Mini Fridge", "price": 40 }, { "title": "Refrigerator", "price": 120 }]</pre>
Verification	• Response contains 2 listings • Both belong to category “Appliances” • No unrelated listings (e.g., Desk Lamp) are returned • Results are returned as a list, not a single item

Component	Details
Feature Being Tested	Seller Creates a new listing
Test Setup	Listings table initially contains: (id: 1, title: “Desk Lamp”, price: 15, category: Appliances, status: ACTIVE)
Action	POST /api/listings with: { "title": "Mini Fridge", "description": "Used mini fridge in good condition", "price": 40, "category": "Appliances" }
Expected result	API returns status 201 Created and JSON: { "id": 2, "title": "Mini Fridge", "price": 40, "category": "Appliances", "status": "ACTIVE" }
Verification	• GET /api/listings returns both listings: ◦ Desk Lamp ◦ Mini Fridge • Listings table now contains: ◦ (id: 1, title: “Desk Lamp”) ◦ (id: 2, title: “Mini Fridge”)

Component	Details
Feature Being Tested	Buyer sends a message to seller about a listing
Test Setup	Messages table initially contains: (id: 1, conversationId: 10, body: "Hello")
Action	POST /api/messages with: <pre>{ "conversationId": 10, "body": "Hi, is this still available?" }</pre>
Expected result	API returns status 201 created and JSON: <pre>{ "conversationId": 10, "body": "Hi, is this still available?" }</pre>
Verification	• GET /api/messages?conversationId=10 returns 2 messages: ◦ "Hello" ◦ "Hi, is this still available?" • Messages table now contains: ◦ id: 1 → "Hello" ◦ id: 2 → "Hi, is this still available?"

Component	Details
Feature Being Tested	User logs in with valid credentials
Test Setup	Users table initially contains: (id: 1, email: "user@umass.edu", password: hashedPassword)
Action	POST /api/auth/login with: <pre>{ "email": "user@umass.edu", "password": "test123" }</pre>
Expected result	API returns status 200 OK and JSON: <pre>{"token": "<JWT token>", "user": { "id": 1, "email": "user@umass.edu" }}</pre>
Verification	• Response contains a valid JWT token • Returned user email matches "user@umass.edu" • No database changes occur during login • Request fails with incorrect password (e.g., returns 401 Unauthorized)

Component	Details
Feature Being Tested	Buyer reports a listing / blocks a user
Test Setup	Reports table initially contains no report for listing id 2 by user id 5
Action	POST /api/reports with: <pre>{"reporterId": 5, "listingId": 2, "reason": "Spam listing"}</pre>
Expected result	API returns status 201 Created and JSON: <pre>{"id": 1, "reporterId": 5, "listingId": 2, "reason": "Spam listing"}</pre>
Verification	• GET /api/reports returns the new report • Reports table now contains: ◦ id: 1 → (reporterId: 5, listingId: 2) • No duplicate report is created for the same user + listing (if applicable) • Report is correctly linked to both user and listing

How we propose to distribute the work among group members

Team Members	Contribution
Vidhi Dave (Frontend)	Frontend design and interactive prototype with dummy products, messaging, login, and initial web app page. Some testing
Gayatri (Frontend)	Frontend development for marketplace pages and user centered features, including product browsing, page layout, UI integration, and testing
Ngan Nguyen (Backend)	Product, Category, Auth (JWT), Messaging, Transactions, Photos, Favorites, Testing
Arkar Myint (Backend)	Listings, User, Services, Auth (JWT), Block, Report, Transactions, Favorites, Testing
Manager	Coordination and Review

Thanks!

Do you have any questions?

Team 10

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons, infographics & images by [Freepik](#)